

RAG 基本架构

功能作用

目的：克服大模型的幻觉现象。

幻觉现象：生成内容看似合理但实际上逻辑混乱或与事实相悖。

幻觉原因：

模型本身：

大模型推理生成时是按照概率取值的，随机性不可避免；解码偏差

训练是 **teacher-forcing**，推理没有纠正，错误会累积；曝光偏差（一次预测多个词）

训练数据：

训练语料里存在知识性错误；知识偏差；

语料未考虑到最新事实；知识过时；

数据未涉及到的知识，无法覆盖所有知识；知识边界；

对齐不当，人的主观错误；

基本概念与分类

从外部数据库中检索相关信息作为上下文传递给大模型，以此信息改善生成质量。

知识检索：对输入问题进行编码，从设置的知识库中高效检索出与问题相关的文档，

生成增强：利用检索文档和输入问题生成最终的输出序列；最常见的方式就是通过 **prompt** 的形式。

架构分类：2*2 四种。

大语言模型根据参数进行感知和调节可分为“黑盒”模型和“白盒”模型，闭源为黑盒，开源根据是否可对参数微调既可视白盒也可视为黑盒；

检索器也可以更新与否。

知识检索

知识库构建

数据采集与处理：为构建知识库提供原材料；

数据清洗：清除文本的干扰元素，如特殊字符等；

文本提取

Docx,PDF，Excel 等文件解析提取。

文本分块：

目的：文本分块即将大块切割成较小单元，适配大模型的上下文长度限制；提高匹配准确率，检索精度；

方法	核心逻辑	优点	缺点
固定长度分块	按字符数分割 $O(n)$	简单、高效	易切断语义
递归分块	按分隔符优先级分割 $O(n)$	平衡效率与语义	依赖分隔符设计
语义分块	基于相似度分割 $O(n^2)$	语义完整性最佳	计算成本高
结构分块	基于文档格式分割 $O(\text{文档格式})$	贴合文档逻辑	依赖格式完整性

知识库增强：

伪查询：作为相关文档的键

原理：为每个文档生成多个 "伪查询"（用户可能用来检索该文档的提问），作为文档的索引键。当用户查询与任何伪查询匹配时，即可召回该文档，解决 "用户用词与文档用词不同但语义相关" 的问题。

输出示例：
伪查询：
1. 考拉主要吃什么食物？
2. 树袋熊每天吃多少桉树叶？
3. 考拉喜欢哪种桉树叶？

应用：将文档与伪查询一同存入知识库（如向量数据库），检索时同时匹配用户查询与伪查询，提升召回率。

代码实例：使用 LLM 为文档生成伪查询

```
from transformers import pipeline
# 加载文本生成模型（如 T5-small，实际可用更大模型如 GPT-3.5）
generator = pipeline("text2text-generation", model="t5-small")
def generate_pseudo_queries(document, num_queries=3):
    """为文档生成伪查询"""
    prompt = f"基于以下文档，生成{num_queries}个用户可能会问的问题（每个问题一行）：\n 文档： {document}"
```

```

    result = generator(prompt, max_length=100,
num_return_sequences=1)[0]['generated_text']
    # 分割生成的问题
    pseudo_queries = [q.strip() for q in result.split("\n") if q.strip()]
    return pseudo_queries[:num_queries] # 确保返回指定数量
# 示例文档
document = "考拉，又称树袋熊，主要以桉树叶为食，每天可进食约 500 克。它们对桉树叶的种类有偏好，尤其喜欢蓝桉树叶。"
# 生成伪查询
pseudo_queries = generate_pseudo_queries(document)
print("伪查询：")
for i, q in enumerate(pseudo_queries, 1):
    print(f'{i}. {q}')

```

文档标题生成：优化文档索引标识

原理：为长文档或无明确标题的文档**生成精准标题**，作为**文档的核心标识**，减少检索时的噪声干扰，提升索引效率。

输出示例：

生成标题：考拉的基本特征与生活习性

应用：用生成的标题作为文档的一级索引，结合内容向量进行双层检索（先匹配标题，再匹配内容），提升效率。

代码实例：使用 T5 模型生成文档标题

```

def generate_title(document, max_length=30):
    """为文档生成简洁标题"""
    prompt = f"summarize: {document}" # T5 的"summarize"任务可用于生成标题
    result = generator(prompt, max_length=max_length)[0]['generated_text']
    return result
# 示例文档（无标题）
document = "考拉是澳大利亚特有的树栖有袋动物，体长约 60-85 厘米，体重 4-15 公斤。它们的爪子锋利，适合攀爬，一生大部分时间在树上度过，每天睡眠 18-20 小时。"
# 生成标题
title = generate_title(document)
print(f"生成标题： {title}")

```

查询增强

普通用户的查询可能会比较泛化不够精确，这样检索的时候会导致匹配效果不好，出现检索出来的内容不准确，不完善的情况。为了改善这种困境，提出了查询增强的方法，核心就是通过**不同方法对用户输入原始查询进行各种改编扩写**，**能够从不同角度更广泛地覆盖**知识库的内容。

语义增强：同义改写

常用的方法有语义增强，即把一个问题换成同义的不同语句，

比如原始查询为 考拉的饮食习惯是什么？

改写 1：考拉主要吃什么？

改写 2：考拉的食物有哪些？

改写 3：树袋熊的饮食结构是什么样的？

代码实例：基于同义词替换和 LLM 改写

```
from nltk.corpus import wordnet
import random

def synonym_replacement(sentence):
    """简单同义词替换（辅助改写）"""
    words = sentence.split()
    new_words = []
    for word in words:
        synonyms = wordnet.synsets(word)
        if synonyms:
            # 随机选择一个同义词（简化处理）
            synonym = random.choice(synonyms).lemmas()[0].name()
            new_words.append(synonym)
        else:
            new_words.append(word)
    return " ".join(new_words)

def llm_rewrite(query, num_rewrites=2):
    """用 LLM 生成同义改写"""
    prompt = f"将以下问题改写为{num_rewrites}个意思相同但表达方式不同的问题（每个问题一行）：\n 问题： {query}"
    result = generator(prompt, max_length=100,
num_return_sequences=1)[0]['generated_text']
    rewrites = [r.strip() for r in result.split("\n") if r.strip()]
    return rewrites[:num_rewrites]
```

多视角分解：分而治之复杂查询

另一种方法则是语义拆解，把一个大问题拆分成多个小问题，用不同的视角提出问题，分而治之，

比如原始查询：考拉面临哪些威胁

子查询 1：自然灾害对考拉的影响有哪些？

子查询 2：外来物种入侵者对考拉有哪些危害？

代码实例：用 LLM 分解查询

```
def decompose_query(original_query, num_subqueries=2):
    """将原始查询分解为子查询"""
    prompt = f"将以下问题分解为{num_subqueries}个更具体的子问题（每个子问题一行），以便更全面地回答：\n 问题： {original_query}"
```

```
result = generator(prompt, max_length=150,
num_return_sequences=1)[0]['generated_text']
subqueries = [s.strip() for s in result.split("\n") if s.strip()]
return subqueries[:num_subqueries]
```

背景文档生成：扩展查询的知识边界

另外一种就是更强的查询增强，即根据查询内容生成一个背景文档，增大覆盖范围，再结合问题和背景文档去检索，更广泛地查询知识库信息。

比如原始问题是考拉的繁殖周期是多久？可以扩展成

背景文档：考拉是有袋类动物，繁殖过程具有独特性。雌性考拉通常在 2-3 岁时达到性成熟，繁殖季节主要集中在每年的春季和夏季.....。

代码实例：用 LLM 生成背景文档

```
def generate_background(query, max_length=200):
    """基于查询生成背景文档"""
    prompt = f"基于以下问题，生成一段相关的背景知识（用于辅助理解问题）：\n 问题： {query}\n 背景知识： "
    result = generator(prompt, max_length=max_length)[0]['generated_text']
    return result
```

文本检索：

文本检索就是根据查询问题及其增强信息匹配知识库里的文本，怎么匹配呢？本质来说就是看查询问题和文本的相似性。

语义检索器即对问题和文本使用两个独立的但相同的模型进行编码，这样创建知识库和检索便可以分开，可以离线创建知识库，通过余弦相似度或内积计算匹配度，适合大规模实时检索。

此外，还有一种常见的传统检索器，即稀疏检索器（如 BM25），这种其实就是关键词匹配。基于 TF-IDF 改进，通过词频、文档长度和逆文档频率计算相关性得分，适合关键词精准匹配。依赖词袋模型，不考虑语义上下文，但**解释性强、速度快**。

优化策略：

混合检索：结合稀疏检索（BM25）进行粗筛，再用双编码器进行语义精排，提升召回率与准确率。

字段权重调整：对标题、实体标签等字段赋予更高权重（如标题权重设为 5），提升关键信息召回。

重排序阶段应用：在双编码器召回的基础上，用交叉编码器对结果进行二次排序，提升精度。

向量数据库优化：采用 FAISS、Milvus 等高效索引工具，支持近似最近邻搜索（ANN），降低检索延迟。

生成增强

把检索到的相关信息和问题组合输入到 prompt 里。

生成增强

利用检索到的信息进行生成增强是一个复杂的过程，不同的方式会显著影响 RAG 的性能。可以从三个角度进行分析：**何时增强**、**何处增强**、**多次增强**。

